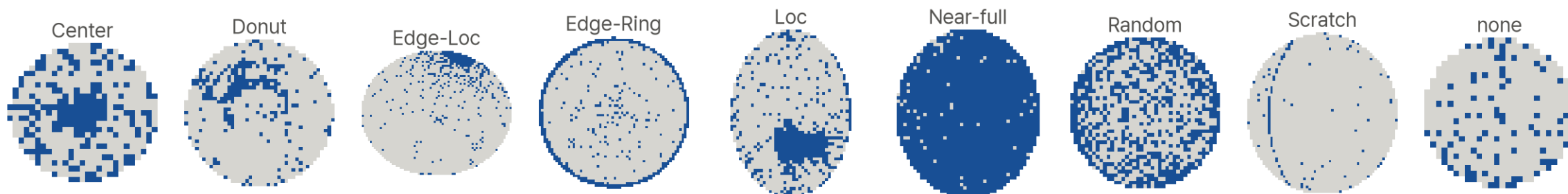


웨이퍼 맵 불량 패턴 분류

lot 단위로 나눈 평가에서 macro-F1 0.909를 확인한 소형 CNN

AI 코딩 에이전트 Claude Code와 역할을 나눠 진행: 판단·검증은 본인, 구현·실험은 AI
WM-811K 공개 데이터 | 개인 프로젝트 | 2026. 9.



이진성

서울과학기술대학교 전기정보공학과 석사과정

01 프로젝트 개요

학습에 쓰지 않은 lot에서 macro-F1 0.909를 확인했습니다.
평가 방식을 먼저 고치고, 결과를 다섯 가지 방법으로 검증했습니다.

기간·형태	2026년 9월, 개인 프로젝트(1인)
데이터	WM-811K 웨이퍼 맵 811,457장 중 라벨 있는 172,950장, 불량 패턴 9종
문제	같은 웨이퍼의 복사본이 학습과 평가에 섞이는 누수를 막고, 새 lot에서의 성능을 재기
결과	macro-F1 0.843(베이스라인) → 0.909(CNN), 모든 패턴 recall 0.77 이상
역할	문제 정의·평가 설계·판단·검증: 본인 / 코드 구현·실험·문헌 조사: Claude Code(AI 코딩 에이전트)
AI 활용	Claude Code(구현·실험) + 별도 검증 에이전트 45개(워크플로 10회). 체크포인트 6개에서 판단 13건, 오류 15건 기록
환경	Python 3.12, PyTorch 2.7(CUDA 11.8), scikit-learn 1.9.1, Streamlit, Quadro P2000

lot 같은 공정 조건으로 함께 처리한 웨이퍼 묶음(최대 25장). 한 lot의 맵은 서로 닮아서 분할의 기본 단위로 씀

macro-F1 패턴 9종 F1의 단순 평균. 85%를 차지하는 none에 점수가 치우치지 않음

recall 실제 그 패턴인 웨이퍼 중 맞힌 비율

진행 순서

1 데이터 탐색과 누수 확인	3쪽
2 평가 설계: lot 단위 분할, 성공 기준	3쪽
3 베이스라인과 소형 CNN	4쪽
4 검증 다섯 가지와 오류 수정	5쪽
5 AI 활용 방식: 지시와 역할	6쪽
6 AI 활용 방식: 검증과 오류 적발	7쪽
7 결론·한계·재현 정보	8쪽

02 문제 정의: 데이터와 평가 방식

원본 분할과 무작위 분할은 같은 웨이퍼의 복사본이 학습과 평가 양쪽에 들어가 점수가 부풀려집니다.
그래서 lot과 복사본을 한 묶음으로 나눴습니다.

누수 3점

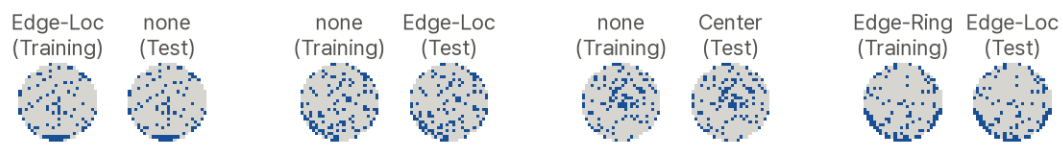
원본 분할	test 3,158장이 train의 복사본. 같은 맵에 다른 라벨이 붙은 쌍 14개
lot 단위 분할	재검사로 생긴 거의 같은 맵이 양쪽에 남음. Center 패턴은 test의 40.7%
반전·회전 복사본	뒤집거나 돌린 복사본도 있어 8가지 변환을 모두 비교해서 묶음

- 데이터 라벨 있는 172,950장 사용.
none 85.2%, Near-full 149장(0.09%)으로 불균형
- 채택한 분할 lot과 10픽셀 이내 복사본(반전·회전 포함)을 한 묶음으로 나눔.
학습:검증:테스트 = 70:15:15
- 성공 기준 test macro-F1 0.80 이상, 모든 패턴 recall 0.50 이상,
베이스라인 초과. 95% 신뢰구간 병기

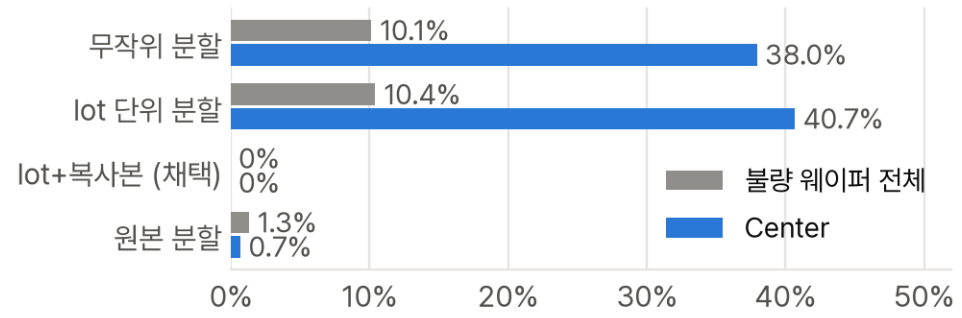
Claude Code 탐색·정제·분할 스크립트 3개 구현, 이상점 탐색 에이전트 10개 병렬 실행

본인 정제 규칙, 분할 방식, 성공 기준 결정(판단 지점 6·7).
라벨 없는 638,507장 제외 결정

같은 맵, 다른 라벨: 원본 Training/Test 사이 복사본 14쌍 중 4쌍



test 웨이퍼 중 학습 쪽에 10픽셀 이내 복사본이 있는 비율



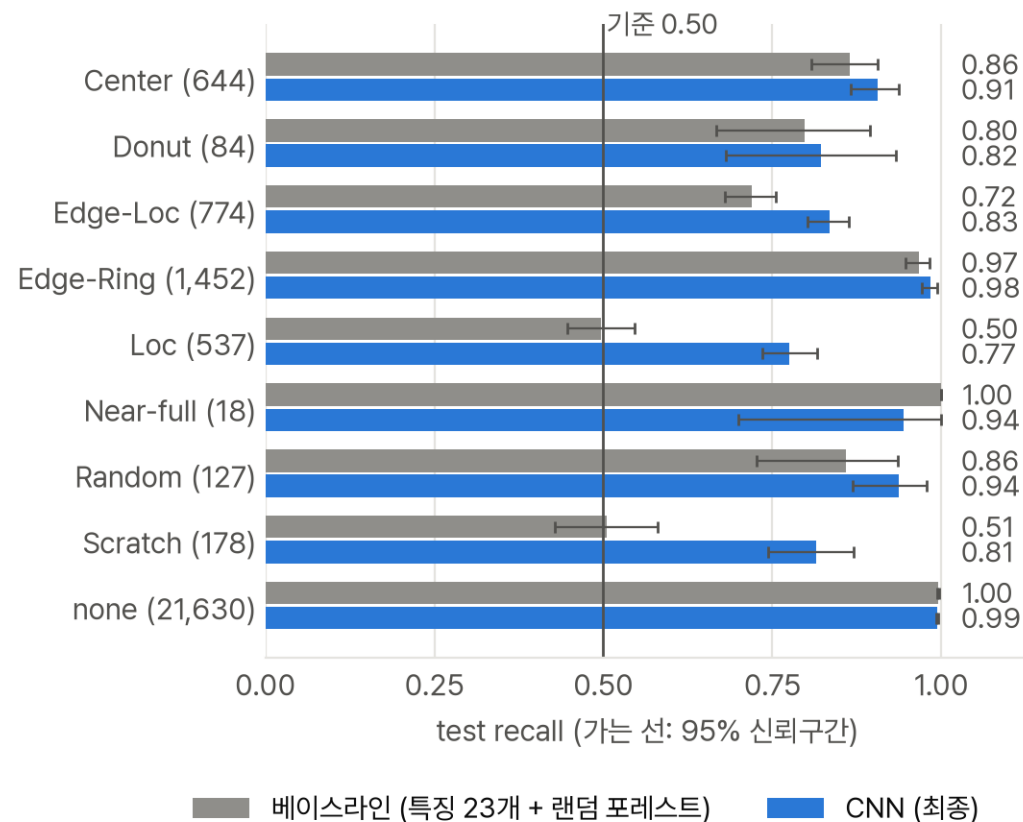
같은 데이터·같은 분할·같은 test 25,444장에서 소형 CNN이 베이스라인보다 macro-F1 0.066 높았고, Loc-Scratch recall이 가장 크게 올랐습니다.

모델	macro-F1 (95% CI)	최저 recall
베이스라인: 특징 23개 + 랜덤 포레스트	0.843 (0.820–0.858)	0.497 (Loc)
소형 CNN, CE	0.901 (0.863–0.918)	0.763 (Loc)
소형 CNN, CE + val 보정 (최종)	0.909 (0.885–0.922)	0.775 (Loc)

입력	64×64로 축소. 축소할 때 블록 최대값을 써서 1다이 폭 Scratch를 남김. 반전·회전 8종 증강
모델	합성곱 블록 4개, 파라미터 58만, CE 20 epoch(GPU 40분). 패턴별 점수 보정값은 val에서만 정함
효과 없던 시도	class-weighted CE: macro-F1 0.825, none 오탐 증가 → 미채택
용도	엔지니어가 어떤 웨이퍼를 먼저 볼지 정하는 1차 분류 보조. 자동 폐기 판정에는 부적합

Claude Code 특징 23개·랜덤 포레스트·CNN 구현, 학습 5회(CE, weighted CE, 셔플 라벨, 무작위 분할, 재학습)

본인 개선 방향과 입력 크기 결정(판단 지점 8),
weighted CE 폐기와 val 보정 채택(판단 지점 10)

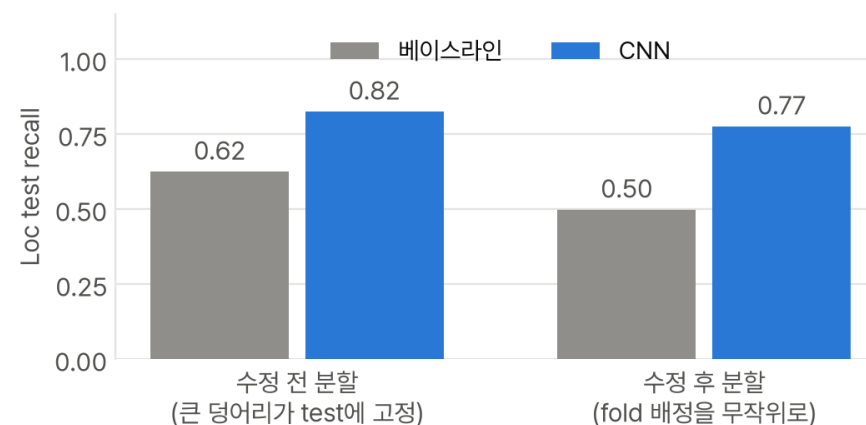


점수가 예상보다 높게 나온 이유를 확인하려고 다섯 가지를 검증했고, 그 과정에서 분할 절차의 오류 하나를 찾아 고쳤습니다.

검증	방법	결과
우연 수준	라벨을 섞어 같은 CNN 학습	macro-F1 0.102 (우연 수준)
누수 제거 효과	무작위 분할 vs lot 분할, 같은 모델	0.895 vs 0.909, 차이 없음
지름길 학습	25×27 제품 외곽선을 다른 제품 것으로 교체	예측 99.5% 이상 유지
오분류 검토	96장을 라벨 가린 채 독립 판정 2회	애매 46% / 모델 오류 27% / 라벨 의심 27%
재현	새 폴더에 clone 후 README 순서로 재실행	1시간 8분, 비교 항목 10개 일치

Claude Code 검증 에이전트 4개 병렬(지표 재계산, 짝비교, 지름길, 이중맹검), 개입 실험·재현 비교 스크립트 구현

본인 오류 판정과 절차 수정 후 전부 재학습 결정(판단 지점 9), 재현 범위 결정(판단 지점 11·12)



분할 절차 오류 (LOG E11)

- 원인** StratifiedGroupKFold(scikit-learn 1.9.1)가 크고 치우친 그룹을 seed와 무관하게 같은 fold에 배정 → 맞히기 쉬운 Loc 웨이퍼 132장이 항상 test에 들어감
- 발견** 베이스라인 점수가 이유 없이 오름 → seed 10개로 다시 나눠 Loc recall 최대 15%p 차이 확인 → lot 묶음 하나를 test에서 빼 보는 실험
- 수정** fold 배정을 seed로 무작위 치환하고 모든 모델 재학습(0.893 → 0.909). 분할 직후 겹침과 lot 묶음 비중을 검사하는 단계 추가

05 AI 활용 방식 ① 지시와 역할

첫 지시문에 AI가 할 일과 멈출 지점, 수치와 오류의 기록 규칙을 정해 두고, 체크포인트 6개에서 판단 13건을 내렸습니다.

첫 지시문에서 정한 규칙 (원문)

“네가 한다: 데이터 탐색, 코드 구현, 실험 실행, 결과 정리, 선택지와 근거 제시”

“판단이 필요한 지점에서는 멈추고 나에게 묻는다. 선택지 2~3개와 각각의 장단점을 제시하고, 추천이 있으면 이유와 함께 밝힌다. 조용히 하나를 골라 진행하지 않는다.”

“내 지시가 틀렸다고 생각하면 반대 의견임을 명시하고 근거를 낸다. 그대로 따르지 않는다.”

“숫자는 실험 결과에서만 가져온다. 추정치를 결과처럼 쓰지 않는다. 결과를 적을 때는 어떤 스크립트·어떤 설정에서 나온 값인지 함께 적는다.”

“오류 기록: 네 코드나 판단이 틀렸던 경우, 누가(나/너) 어떻게 발견했고 무엇을 바꿨는지. 틀린 것을 숨기지 말고 반드시 남긴다.”

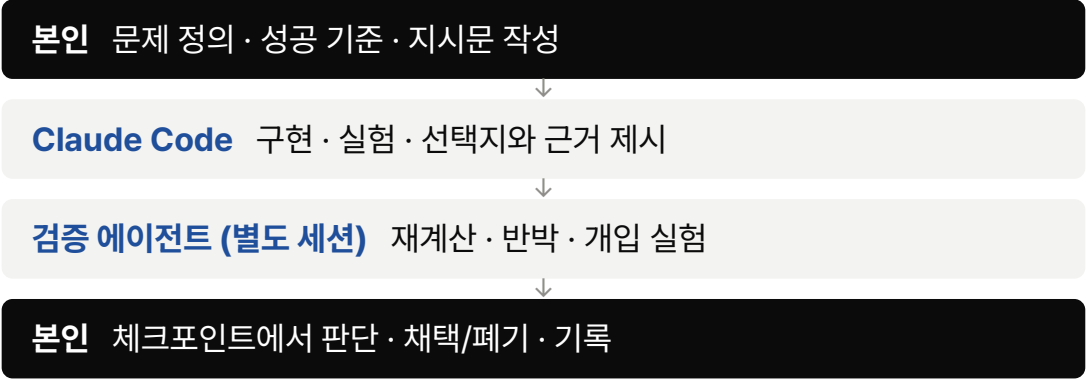
“각 체크포인트(★)에서는 반드시 멈추고 나에게 보고한 뒤 지시를 기다린다.”

진행 중 지시 (채팅 원문)

“내가 100프로 만족할 수 있게 나에게 질문하면서 진행해” 첫 지시. 체크포인트마다 멈추게 한 근거

“대시보드의 문체를 다른 사이트를 참고해서 바꿔줘 너무 ai틱해” 산출물 반려. 참고 사례를 조사한 뒤 문구를 다시 씀

작업 흐름



체크포인트 6개 · 판단 13건 · 검증 워크플로 10회(에이전트 45개) · 오류 15건 기록

판단	선택	근거
라벨 없는 638,507장	제외	불량 0개 맵이 5.8만 장 등 분포가 다르고, 평가 맵의 복사본 5,408장이 섞여 있음
성공 기준	macro-F1 0.80 + 전 패턴 recall 0.50	선행연구의 lot 분할 0.85는 재검사 복사본 누수 가능성이 크고, 라벨 노이즈도 있음
분할 오류 대응	절차 수정 후 전부 재학습	얕은 맵 묶음 규칙은 유지하고 fold 배정만 무작위로. 이전 결과는 '편향된 분할'로 표시해 남김

AI 활용 방식 ② 검증과 오류 적발

구현한 AI와 별개의 검증 에이전트 45개를 10회 돌려 AI가 낸 오류 15건 중 7건을 잡았고, 나머지 8건은 실행 전 검토와 커밋 전 대조 규칙에서 나왔습니다.

AI가 낸 오류를 잡은 사례

오류	어떻게 드러났나	조치
E10 잘못된 원인 가설	AI가 세운 가설(라벨이 다른 재검사 쌍의 역누수)을 AI가 짠 진단 스크립트가 반증	가설 철회, 원인 조사 워크플로로 전환
E11 분할 절차 오류	반박 검증 에이전트가 scikit-learn 소스와 개입 실험으로 확인	절차 수정, 모든 모델 재학습
E13 확률 열 버그	지표 검증 에이전트가 예측 파일로 보정 결과를 재현하다 발견	저장 로직 수정 후 재평가, 지표 변화 없음

독립 검증·조사 워크플로 10회 (에이전트 45개)

단계	에이전트	한 일
데이터 확보·문헌 조사	10	조사 5 + 반박 검증 5. 오류 E1~E4 발견
데이터 이상점 탐색	10	탐색 4 + 검증. 부분 정정 7건(E7)
베이스라인·분할 재계산	4	지표 재계산, 독립 재구현, 코드 감사
Loc recall 원인 조사	3	원인 추적·구성 점검·반박 → E11
CNN 검증 2회	8	지표 재계산, 짝비교, 지름길, 이중맹검 → E13
문체·포트폴리오 조사	10	대시보드 문구 4, 포트폴리오 3 + 3

산출물 반려

산출물	문제	조치
대시보드 문구	AI가 쓴 티: 한단체 설명문, 이모지, 내부 실행 이름	수율 도구·ML 리포트·UX 라이팅 관례 조사 후 재작성
포트폴리오 1차	과밀, 반말 서술 제목, 쪽별 주장 파악 안 됨	실제 국내 텍 6건 조사 후 8쪽으로 재구성

규칙

구현한 에이전트와 검증하는 에이전트를 분리. 검증 쪽에는 원본 코드 대신 데이터와 결과 파일만 주고 다시 계산하게 함

나머지 8건

실행 전 자체 검토, 그림·표를 직접 열어 확인, 커밋 전 grep 대조, 내보내기 후 pdffonts 점검에서 발견(E5·E6·E8·E9·E12·E14·E15)

07 결론·한계·재현 정보

이 결과는 같은 제품의 새 lot에 대한 1차 분류 보조 수준입니다.
수율 개선이나 공정 원인 규명 효과는 주장하지 않습니다.

결론

- 누수를 막은 평가에서 CNN 0.909, 베이스라인 0.843
- 분할 절차 하나로 Loc recall이 12%p 달라짐. 모델을 고르기 전에 평가 방식부터 확인해야 함
- 검증 5종 통과, 새 폴더 재현 일치

한계·다음 단계

- 한 번 나눈 분할의 결과. seed 10개로 나눠 평균하면 약 0.01 낮음 → 그룹 5-fold로 전체 평가
- Near-full test 18장, 신뢰구간 0.70~1.00 → 표본 확보 전까지 수치 유보
- 같은 맵 다른 라벨 14쌍, 6~10다이 짧은 Scratch 약점 → 라벨 기준 재정의, 고해상도 입력

배운 점

- 수치는 스크립트 실행 결과에서만 옮긴다. 추정치는 쓰지 않는다
- 점수가 갑자기 오르면 원인을 찾기 전까지 채택하지 않는다
- 새 데이터를 받으면 모델을 만들기 전에 분할 방식과 라벨 품질부터 검사한다

재현 정보

환경	Windows 11, Python 3.12.6, torch 2.7.1+cu118, scikit-learn 1.9.1, Quadro P2000. 시드 0 고정
실행	python scripts/run_all.py (데이터 다운로드 → 분할 → 베이스라인 → CNN → 평가, 약 4~5시간). 결과 outputs/*.json
저장소	github.com/sl2ver/wm811k-wafer-defect-classification (MIT). PLAN.md 계획·변경 이력 / LOG.md 지시 원문·판단 13건·오류 15건 / RESULT.md 수치·버린 것 9·한계 9